



TRABAJO FIN DE MÁSTER

Universidad Politécnica de Madrid



Máster Universitario en
FORMACIÓN DEL PROFESORADO DE EDUCACIÓN SECUNDARIA
OBLIGATORIA, BACHILLERATO y FORMACIÓN PROFESIONAL

DESARROLLO DE COMPETENCIAS STEM MEDIANTE SCRATCH

Nombre: **Margarita Domingo Blázquez**

Curso: **2015-2016**

Especialidad: **Tecnología**



TRABAJO FIN DE MÁSTER

Universidad Politécnica de Madrid



Máster Universitario en
FORMACIÓN DEL PROFESORADO DE EDUCACIÓN SECUNDARIA
OBLIGATORIA, BACHILLERATO y FORMACIÓN PROFESIONAL

DESARROLLO DE COMPETENCIAS STEM MEDIANTE SCRATCH

Nombre: **Margarita Domingo Blázquez**
Curso: **2015-2016**
Especialidad: **Tecnología**
Dirección: **José Luis Martín Núñez**
Instituto de Ciencias de la Educación (ICE)
Línea temática: **Tecnologías educativas**





TRABAJO FIN DE MÁSTER

Universidad Politécnica de Madrid



Máster Universitario en
FORMACIÓN DEL PROFESORADO DE EDUCACIÓN SECUNDARIA
OBLIGATORIA, BACHILLERATO y FORMACIÓN PROFESIONAL



ÍNDICE

1	INTRODUCCIÓN.....	9
2	MARCO TEÓRICO	11
2.1	COMPETENCIAS STEM	11
2.2	PROGRAMACIÓN	13
2.3	SCRATCH	17
3	PROPUESTA METODOLÓGICA	21
3.1	OBJETIVOS	21
3.2	INSTRUMENTOS.....	21
3.3	CUESTIONARIOS.....	22
3.4	MUESTRA.....	25
3.5	PROCEDIMIENTO	25
4	RESULTADOS.....	27
4.1	RAZONAMIENTO ABSTRACTO (RA).....	27
4.2	APTITUD ESPACIAL (APE)	29
4.3	APTITUD NUMÉRICA (APN)	32
5	CONCLUSIONES	35
5.1	ANÁLISIS DE LA EXPERIENCIA	35
5.2	CONCLUSIONES FINALES	39
6	REFERENCIAS BIBLIOGRÁFICAS	40
7	ANEXOS	43
7.1	UNIDAD DIDÁCTICA	43

RESUMEN

En este trabajo se van a evaluar competencias STEM en alumnos de la etapa de Educación Secundaria tras el aprendizaje de varios conceptos de informática y principalmente programación, siendo Scratch el entorno de programación por bloques utilizado. Las competencias STEM analizadas han sido el razonamiento lógico-matemático, la resolución de problemas, la estructuración y desarrollo de ideas y la visualización gráfica y espacial. Para realizar la evaluación de las competencias, en primer lugar se ha realizado a los alumnos un test que mide la inteligencia general y factorial, posteriormente han cursado la unidad didáctica de programación y finalmente se les ha vuelto a realizar el mismo test. Los resultados del estudio han sido favorables y se puede concluir que tras el aprendizaje de la programación con Scratch mejoran las competencias STEM.

Palabras clave: competencias, STEM, informática, programación, Scratch, razonamiento lógico, razonamiento matemático, resolución de problemas, estructuración de ideas, visualización gráfica, visualización espacial

ABSTRACT

This paper will assess STEM competences in Secondary School students, after learning some concepts of computers and mainly programming. In particular, this paper is focused in Scratch as programming environment with blocks. Logical-mathematical reasoning, problem solving, structuring and development of ideas and graphical and spatial visualization are the STEM competences studied in this report. So as to assess these competences, firstly students did a test of general and factorial intelligence, secondly they studied about programming and Scratch and finally they repeated the same test of intelligence. The results have been positive and we can conclude that learning programming can improve STEM competences.

Keywords: competences, STEM, computers, programming, Scratch, logical reasoning, mathematical reasoning, problem solving, structuring of ideas, graphical visualization, spatial visualization

1 Introducción

Este documento es un Trabajo Fin de Máster realizado en el Máster Universitario en Formación del Profesorado de Educación Secundaria Obligatoria, Bachillerado y Formación Profesional de la Universidad Politécnica de Madrid, durante el curso 2015-2016.

En este trabajo se presenta un estudio con el fin de evaluar competencias STEM, del inglés Science, Technology, Engineering y Mathematics, en el aprendizaje de la programación, concretamente a través de la realización de un juego desarrollado en Scratch, que es un entorno de programación gráfico por bloques. Dicho juego se propone en la asignatura de Tecnología, Programación y Robótica de 1º ESO, en la unidad didáctica de Programación.

Y pretende demostrar que, con la programación, es posible mejorar ciertos aspectos relacionados con dichas competencias:

- Razonamiento lógico-matemático.
- Resolución de problemas y depuración.
- Estructuración de ideas o desarrollo de ideas a partir de una concepción inicial de proyecto.
- Visualización gráfica y espacial.

El trabajo se presenta en varios capítulos.

En el primero de ellos se muestra el marco teórico, es decir, muestra el punto de partida del proyecto. Se refiere a estudios, investigaciones y documentos realizados hasta la fecha y utilizados para la elaboración de este estudio. Y se desglosan en tres apartados, el marco teórico relacionado con competencias STEM, el relacionado con la programación y el relacionado con Scratch.

En el segundo capítulo se habla de la propuesta metodológica. Dentro de este capítulo se marcan los objetivos de este trabajo y los medios utilizados para realizarlo, donde se incluyen los instrumentos, los cuestionarios usados para analizar la evaluación de competencias STEM, la muestra y el procedimiento llevado a cabo.

El tercer capítulo muestra los resultados obtenidos del estudio, comprobando si realmente se mejora competencias STEM usando Scratch como método inicial de la programación.

Por último, en el cuarto capítulo se recopilan las conclusiones obtenidas tras el estudio realizado.

Al final del documento, aparece la lista de referencias bibliográficas utilizadas para el desarrollo de este trabajo.

2 Marco teórico

Cada vez son más los países que están incorporando una formación académica en informática dentro de los estudios de la enseñanza secundaria y bachillerato. En el caso de España y, concretamente, de la Comunidad de Madrid, se ha modificado la antigua asignatura de Tecnología por una nueva asignatura llamada Tecnología, Programación y Robótica (Gobierno de la Comunidad de Madrid, 2015). Es por ello por lo que parece relevante hablar de competencias STEM, de la programación en general dentro de las aulas y concretamente de Scratch, uno de los programas más utilizados para empezar a programar.

2.1 Competencias STEM

El término STEM son las siglas en inglés de Science, Technology, Engineering y Mathematics (Ciencias, Tecnología, Ingeniería y Matemáticas). Surgió hace 120 años en el Comité de los Diez en Harvard, al querer modernizar el sistema escolar dentro de una sociedad cada vez más industrial y menos agraria, con más tecnología en general (Lloyd, 2012).

Con las habilidades STEM se pretende desarrollar el emprendimiento, la innovación y la competitividad, básicos dentro de la economía actual, con el fin de mejorar la inserción en el mundo laboral de esta sociedad cada vez más tecnológica (Uzcanga y Gómez, 2015). Hasta ahora se han enseñado matemáticas y ciencias de forma aislada, pero el nuevo concepto persigue que se enseñe junto, en un solo bloque y además añadiendo la ingeniería y la tecnología.

Una de las principales razones por las que se intentan promover las habilidades STEM es que se ha comprobado que cada vez hay una brecha mayor entre el número de profesionales necesarios en el sector STEM y el número de personas que se preparan profesionalmente en este sector. Así lo afirma Randstad en una nota de prensa con el siguiente titular: “España será uno de los países europeos con mayor desajuste entre la formación de sus profesionales y las necesidades empresariales”. Y es que, según este informe, las previsiones apuntan a que “en 2020 faltarán 1,9 millones de profesionales altamente cualificados” (Randstad, 2016).

Analizando el déficit de talento por continentes, se aprecia que habrá desajustes a nivel mundial, afectando principalmente a Norteamérica, Asia y Europa, según se muestra en el siguiente gráfico:

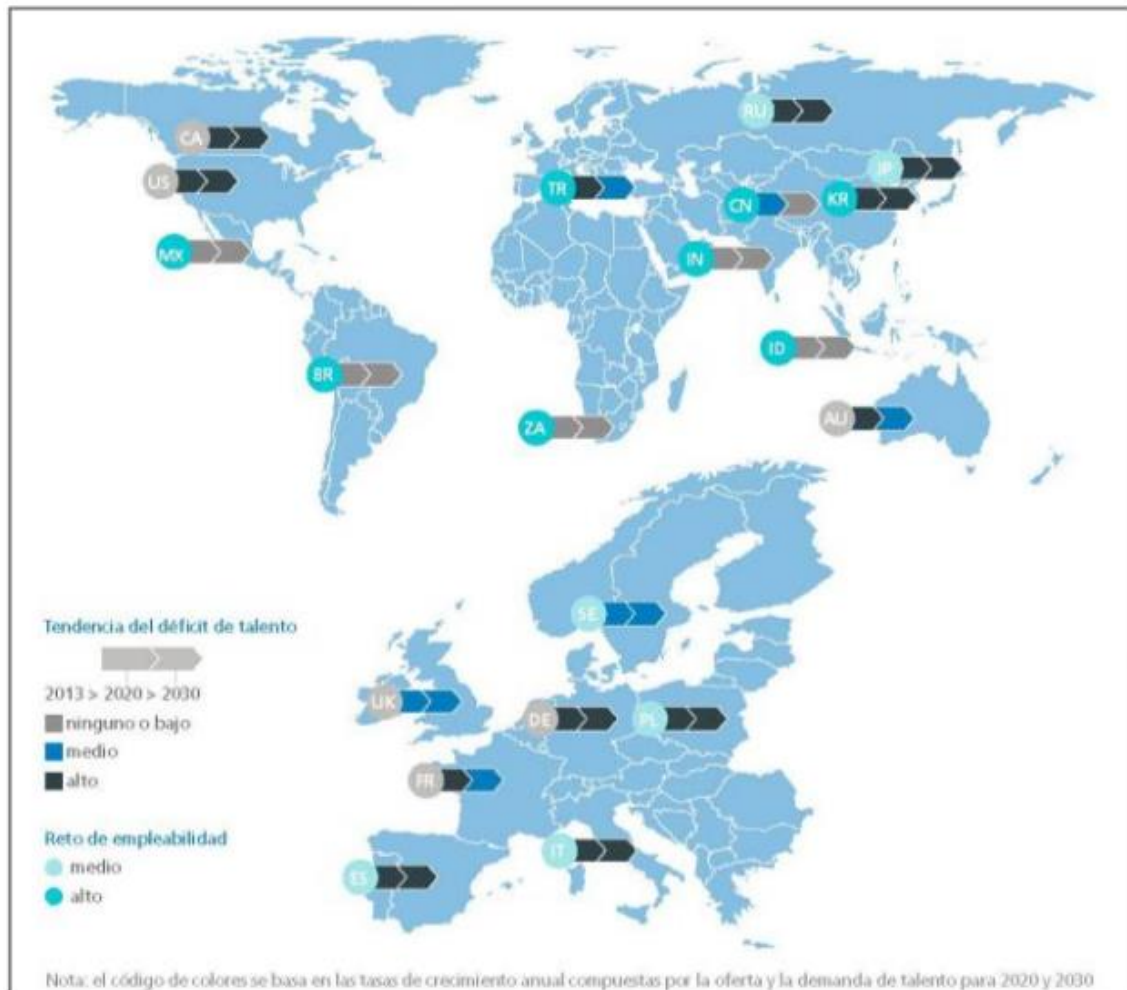


Ilustración 1. Déficit de talento, escasez de trabajadores altamente cualificados para 2020 y 2030 (Randstad, 2016)

Para fomentar las vocaciones tecnológicas cada vez se están uniendo más países a proyectos cuyo objetivo en educación es desarrollar las competencias STEM. Así lo muestra una publicación de la Fundación Telefónica donde se recogen las 100 mejores innovaciones educativas basadas en las habilidades STEM que se han llevado a cabo en diversos países con resultados demostrados (Fundación Telefónica, 2014). Estas innovaciones pretenden dotar a los alumnos de la información, las habilidades y los conocimientos necesarios para fomentar su vocación por los estudios STEM.

Dentro de las competencias STEM aparece un nuevo concepto que ayuda a incrementar la vocación tecnológica relacionada con la educación STEM (Vásquez, 2015). Se trata del pensamiento computacional.

El pensamiento computacional lo define Jeannette Wing (Wing, 2006), gran promotora de dicho pensamiento, como la automatización de abstracciones, es decir, se refiere al proceso de pensamiento implicado en la formulación de un problema y en su solución de manera que una computadora -humano o máquina- pueda llevar a cabo con eficacia. Eso no significa simplemente programar, sino que pensar como un informático requiere varios niveles de abstracción y una buena abstracción supone: eficiencia, corrección, simplicidad, escalabilidad, usabilidad, mantenimiento, coste,...

Por lo que podemos decir que el término de pensamiento computacional incluye (Quesada, 2015):

- La comprensión y estructuración de los problemas.
- La reformulación lógica de los aspectos clave de un problema.
- La descomposición del problema en partes.
- El diseño de procesos sistemáticos para la resolución de problemas.
- La generalización de las soluciones.
- El estudio y comparación de las soluciones: corrección, eficiencia,...

En este estudio analizaremos el impacto de la programación Scratch en el pensamiento computacional, dentro de las competencias STEM. Y, concretamente, analizaremos el impacto de la programación en el razonamiento lógico-matemático, la resolución de problemas, la estructuración de ideas y la visualización gráfica y espacial.

2.2 Programación

Es evidente la presencia del mundo informático dentro de la vida cotidiana de toda la sociedad. El uso de móviles, tablets u ordenadores (mayoritariamente portátiles) y el uso de internet comienza cada vez a edades más tempranas.

Según la siguiente tabla obtenida de una encuesta sobre equipamiento y uso de Tecnologías de Información y Comunicación en los Hogares de octubre de 2015 (Instituto Nacional de Estadística, 2015) realizada por el Instituto Nacional de Estadística, más del 95% de los jóvenes españoles entre 10 y 15 años usan el ordenador y el 93,6% utilizan internet:

Porcentaje de menores usuarios de TIC por sexo y edad Año 2015

	Uso de ordenador	Uso de Internet	Disposición de móvil
Total	95,1	93,6	67,0
Sexo			
Hombres	95,3	93,0	64,4
Mujeres	94,9	94,2	69,8
Edad			
10	90,8	88,0	29,7
11	93,9	94,5	42,2
12	96,6	92,0	69,5
13	96,1	96,2	78,4
14	96,5	95,8	90,4
15	96,3	94,8	90,9

Ilustración 2. Porcentaje de menores usuarios de TIC (Instituto Nacional de Estadística, 2015)

Pero que los jóvenes de nuestro país utilicen el ordenador no significa que lo usen correctamente, ya que la mayoría de ellos lo utilizan como entretenimiento principalmente en las redes sociales. Según la misma encuesta comentada anteriormente pero realizada un año antes, en octubre de 2014, (Instituto Nacional de Estadística, 2014) un 92% de los estudiantes utilizaban redes sociales. Muy probablemente esta cifra haya aumentado actualmente, porque ya hace un año y medio de este estudio. Esta situación no es algo particular en España, sino que es a nivel mundial.

Se considera a Alan Turing el padre de la ciencia informática, cuando en 1936 creó la Máquina de Turing (Wikipedia, 2016). La ciencia informática se consideraba entonces una combinación de matemáticas y de ingeniería eléctrica. Pero desde entonces la informática ha evolucionado a pasos agigantados y cada vez tiene más aspectos como:

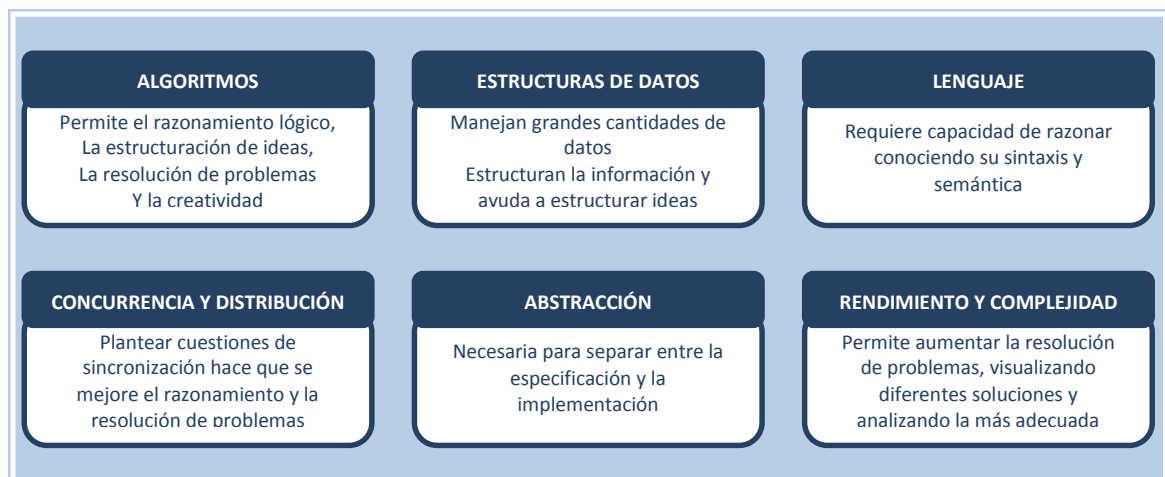


Ilustración 3. Aspectos generales de la informática. Fuente: Elaboración propia a partir de (Gander et al., 2013)

Quizá no sea necesario enseñar todo esto en niveles de secundaria, pero sí tener unos conceptos mínimos sobre la informática. Teniendo en cuenta que antes de llegar a la Universidad ya tenemos unos conocimientos básicos en matemáticas o en física o en biología, sería lógico pensar que deberíamos tener unos conocimientos básicos en informática, aunque la carrera que vayamos a realizar no sea la de Ingeniería Informática. Así podemos encontrarlo en un manifiesto para la enseñanza de la informática del siglo XXI (Naughton, 2012)

Varias organizaciones y asociaciones coinciden en que debería darse una formación académica de informática en cursos de niveles inferiores a la Universidad. Y también coinciden en que uno de los principales inconvenientes es la escasez de maestros cualificados que puedan enseñar a los niños. Por ejemplo, en junio de 2014, la Conferencia de Directores y Decanos de Ingeniería Informática (CODDII) y la Asociación de Enseñantes Universitarios de la Informática (AENUI) se unieron para formular una Declaración titulada “Por la inclusión de asignaturas específicas de ciencia y tecnología informática en los estudios básicos de la enseñanza secundaria y bachillerato”, donde analizan la situación por la que están pasando la nueva generación, considerada nativa digital y sin ninguna atracción por los estudios relacionados con las TIC, es decir, las Tecnologías de la Información y la Comunicación (CODDII y AENUI, 2015).

Esta situación supone un problema, sobre todo teniendo en cuenta que “la Comisión Europea estima que hacia el año 2020 se podría afrontar una necesidad de disponer de cerca de 900.000 profesionales en el sector de las TIC” (CODDII y AENUI, 2015), por lo que proponen introducir competencias informáticas en edades anteriores a la universidad, con el fin de generar atracción hacia esa rama STEM gracias al conocimiento conseguido.

Para conseguir esta mejoría, esta declaración propone una serie de recomendaciones. Por un lado, se debe ofertar la asignatura de tecnología en todos los centros y todos los estudiantes deberían estudiar ciencia y tecnología informática. Para ello, sería necesario formar al profesorado, ya que las competencias STEM se pueden desarrollar en otras asignaturas de Educación Secundaria y Bachillerato de forma transversal. Por último, proponen realizar actividades o talleres escolares relacionados con la tecnología, con el fin de enseñar y motivar a los alumnos dentro de esta área.

Una de las razones para implantar asignaturas específicas de ciencia y tecnología informática es que las habilidades relacionadas con las habilidades STEM “aportan beneficios como la estructuración de la mente y la mejora en la manera de razonar, útiles para la totalidad del

alumnado independientemente de su itinerario o modalidad académica” (CODDII y AENUI, 2015).

Asociaciones mundiales como ACM, *Association for Computing Machinery*, y CSTA, *Computer Science Teachers Association*, también coinciden en que el mundo que nos rodea y nuestra economía está gobernado por la informática y las tecnologías, por lo que debemos prepararnos para los empleos del siglo XXI y para conseguirlo “los estudiantes deben tener una comprensión más profunda de los fundamentos de la informática” (ACM y CSTA, 2010).

El gobierno de Reino Unido nos resume las ventajas de incluir la programación dentro del currículo en una publicación del Departamento de Educación ya que la programación tiene un gran vínculo con las competencias STEM y ayuda a los alumnos a mejorar el pensamiento computacional y la creatividad (UK. Department for Education, 2013).

Es por ello por lo que están sufriendo una “revolución tecnológica” en las escuelas (Cano y Delgado, 2015), subvencionando la compra de placas Raspberry Pi y Arduino y además están promoviendo campamentos y cursos de programación para adolescentes y otras iniciativas como CAS o *Computing at School* (BCS The Chartered Institute for IT, 2016), que es una comunidad de padres que quieren dar a sus hijos una formación académica en informática más avanzada.

Otro informe de ACM, *Association for Computing Machinery*, junto con *Informatics Europe*, una asociación europea de institutos de investigación académica e industrial en el campo de ciencias de la información y la informática, también han emitido informes que manifiestan la necesidad de incluir una “alfabetización digital” en las escuelas de Europa (Gander et al., 2013).

En España, con el cambio de ley a la LOMCE, Ley Orgánica para la Mejora de la Calidad Educativa, durante este curso 2015-2016 (Gobierno de España, 2013), ya se han incorporado algunos cambios en la asignatura de Tecnología de 1º y 3º de la ESO, donde se incluye una unidad didáctica de Programación y de Robótica. Para el próximo curso 2016-2017 se incluirá la asignatura de Tecnología en 2º y 4º de la ESO y se espera que también incluya una parte relacionada con la informática y la programación. Poco a poco se va avanzando en el ámbito informático, aunque todavía queda mucho camino por recorrer.

Entre las mejoras a nivel mundial en el ámbito de la programación aparecen Codecademy, una empresa de educación que ofrece clases de programación. Y code.org, que es un movimiento global apoyado por figuras clave de la industria tecnológica como Bill Gates, fundador de Microsoft, o Mark Zuckerberg, creador de Facebook, que busca mostrar a alumnos y profesores las ventajas de enseñar programación en las escuelas. Desde su página quiere enseñar a

programar con cursos desde los 4 años de edad, porque consideran que “al introducir al estudiante tempranamente en las ciencias computacionales tendrán una base para el éxito en cualquier carrera del siglo XXI” (Cano y Delgado, 2015).

En resumen, son varios los estudios que confirman que la programación en el aula permite mejorar el aprendizaje de asignaturas como matemáticas, lengua o ciencias y el desarrollo de habilidades como la resolución de problemas, el pensamiento lógico y la creatividad (Meseguer et al., 2015).

2.3 Scratch

Scratch es un entorno gráfico con bloques de código abierto creado por el Instituto Tecnológico de Massachusetts (MIT). Está especialmente indicado para personas que se están iniciando en el mundo de la programación, ya que es muy sencillo de utilizar y muy intuitivo (MIT, 2016).

Además de ser gráfico, Scratch es muy atractivo y sencillo de usar. Permite realizar programas, como juegos o aplicaciones de forma muy sencilla.

Por un lado tiene la parte gráfica, para realizar el diseño de la aplicación, donde el usuario puede crear el fondo y los objetos que van a aparecer en el programa. Cada objeto tiene disfraces y sonidos asociados y el usuario puede crear, modificar o borrar objetos y los disfraces y sonidos que tengan asociados, con el fin de utilizarlos en diferentes puntos del programa.

Por otro lado tiene la parte de codificación asociada a cada objeto y al escenario. Para ello utiliza bloques que Scratch ya tiene definidos y agrupados por categorías, asociando un color a cada una de las categorías para hacerlo más visual. Estos bloques son instrucciones sencillas y el programa se construye encajando distintos bloques uno detrás de otro como si de un puzle se tratase.

Además de las ventajas vistas hasta el momento, como la sencillez o que sea muy intuitivo y visual, otra ventaja es que no tiene los problemas de sintaxis que puede haber en otros lenguajes de programación porque los bloques ya están correctamente definidos.

Debido a todas estas razones es un buen método para que los estudiantes más jóvenes puedan iniciarse en la programación.

Al igual que marcan algunos estudios analizados en el apartado de programación, también Mitchel Resnick (Resnick, 2007), uno de los responsables del proyecto Scratch, dice que

aprender a programar es útil para todos aunque todos no lleguen a ser ingenieros informáticos, igual que todos aprendemos a escribir y no todos somos escritores profesionales.

Scratch está diseñado para promover lo que el Laboratorio de Medios del MIT (Life Long Kindergarten) dirigido por Mitchel Resnick, llama “espiral de pensamiento creativo”, donde inicialmente el usuario imagina sus propias ideas (resolviendo problemas con el uso del razonamiento lógico) y las desarrolla (estructura las ideas y las desarrolla a partir de una concepción inicial de proyecto, llevando a cabo una visualización gráfica y espacial), les permite jugar con sus creaciones y compartirlas para que otros también puedan jugar, y posteriormente permite recibir retroalimentaciones de otras personas y reflexionar sobre el trabajo realizado para volver a generar nuevas ideas con la experiencia adquirida en este proceso.

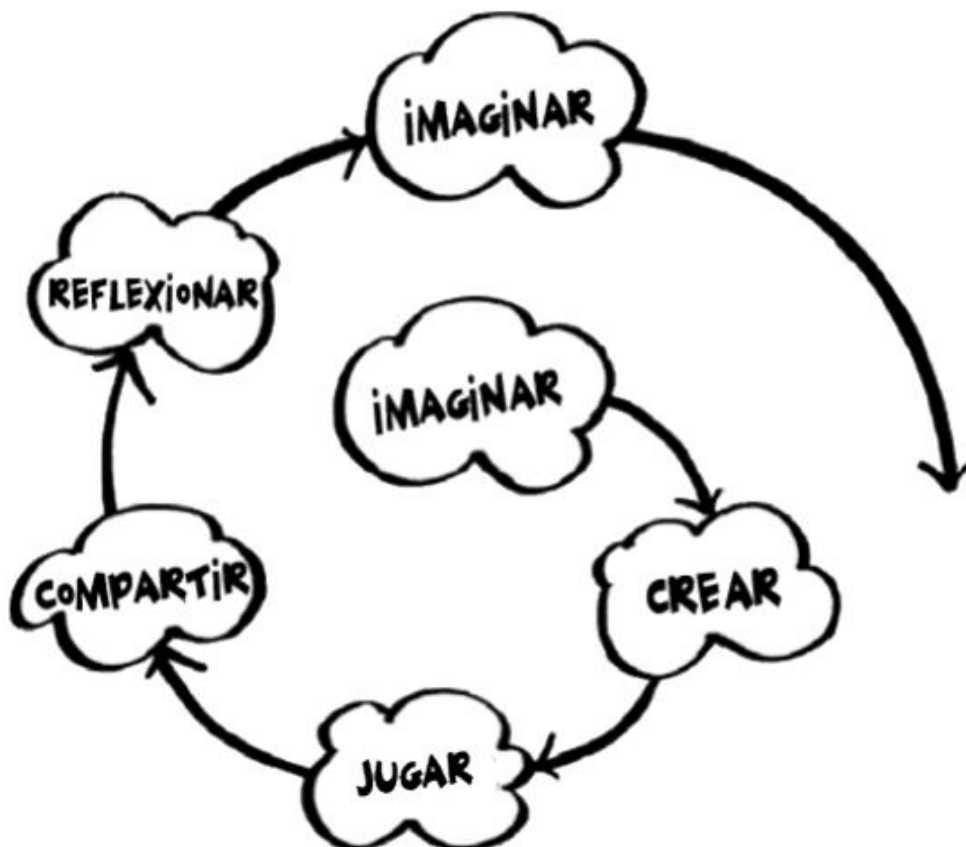


Ilustración 4. Espiral del pensamiento creativo, según Mitchel Resnick (Resnick, 2007)

A modo de resumen, Mitchel Resnick afirma que “para tener éxito en la actual sociedad de la Creatividad, deben aprender a pensar de manera creativa, planear sistemáticamente (estructuración de ideas), analizar críticamente (resolución de problemas), trabajar colaborativamente, comunicarse claramente, diseñar iterativamente y aprender

continuamente. Y todo esto lo pueden aprender con Scratch". Todas estas habilidades son útiles para todos independientemente del trabajo que tengan (Resnick, 2007).

También Pablo Espeso en la revista online educaciontrespuntocero.com (Espeso, 2016), afirma que con el uso de Scratch se pueden promover "habilidades como el razonamiento lógico-matemático, la expresión verbal y lingüística, o todo lo englobado dentro del pensamiento computacional".

Efectivamente, el pensamiento computacional aparece en Scratch en la búsqueda y solución de problemas, ya que crear un proyecto requiere reflexionar sobre los aspectos clave del problema, pensar cómo descomponer el problema en pasos, diseñar el proceso y finalmente implementarlo usando los bloques de programación (Cearreta-Urbieta, 2015).

El equipo de Scratch de la Universidad de Harvard (Resnick y Brennan, 2011) desglosa el pensamiento computacional en 3 puntos basados en las ideas informáticas:

- conceptos (secuencias, bucles, paralelismo, eventos, condicionales, operadores, variables y listas),
- prácticas (incrementar, iterar, probar, depurar, reusar, remezclar, abstraer, modularizar)
- perspectivas (expresión, conectividad y comprensión). A medida que los estudiantes programan y comparten los proyectos de Scratch, van desarrollándose como pensadores computacionales: aprenden conceptos computacionales y matemáticos básicos, estrategias de diseño, resolución de problemas y trabajo en colaboración.

Hay varias publicaciones (Maloney et al., 2004) e instituciones, como Harvard (Brennan y Rusk, 2009), que corroboran que el uso de Scratch proporciona fluidez tecnológica y permite mejorar habilidades como el razonamiento lógico, la resolución y depuración de problemas y el desarrollo de ideas a partir de una concepción inicial del proyecto hasta su finalización.

Y también Sánchez Montoya (Montoya, 2008) tiene un estudio donde muestra las inteligencias que puede desarrollar un alumno con el uso de Scratch, y donde destacamos la lógico-matemática y la espacial, que son dos de las que se tratarán en este proyecto:

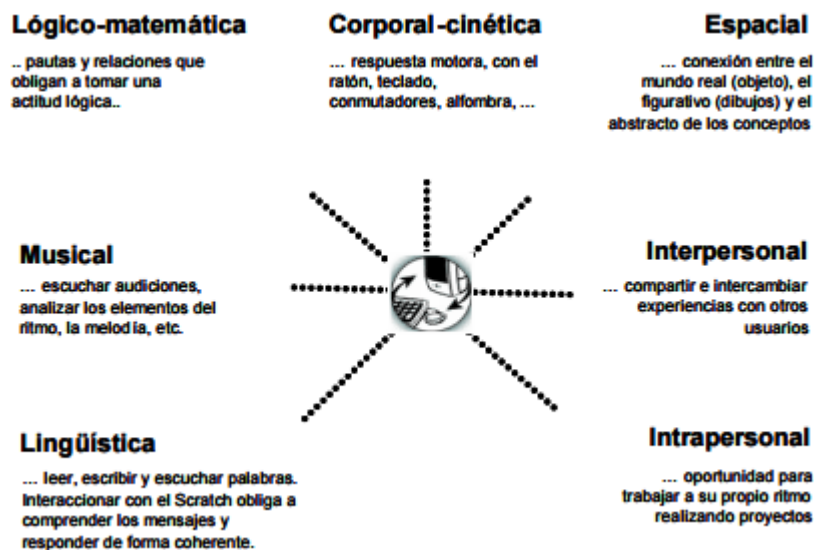


Ilustración 5. Inteligencias que puede desarrollar un alumno con el uso de Scratch (Montoya, 2008)

Por lo que se puede confirmar que “utilizando Scratch se desarrolla el pensamiento lógico matemático para comunicar ideas de número, espacio, formas, patrones y problemas de la vida cotidiana” (Valle y Salgado, 2013).

En resumen, los estudios revisados en este marco teórico muestran que la programación en general y Scratch en particular puedan ayudar a contribuir en el desarrollo de ciertas cualidades del estudiante. En este estudio se analizarán las siguientes:

- Razonamiento lógico-matemático
- Resolución de problemas
- Estructuración de ideas o desarrollo de ideas a partir de una concepción inicial del proyecto
- Visualización gráfica y espacial

Todas ellas incluidas dentro de las competencias STEM.

3 Propuesta metodológica

3.1 Objetivos

El objetivo general de este trabajo fin de máster es evaluar competencias STEM a través de un juego programado en Scratch propuesto en la unidad didáctica de Programación para alumnos de 1º de la ESO. Esta unidad consiste en la elaboración de un proyecto creando algoritmos y estructurando un programa de bloques. Se trabajará con el movimiento de objetos sobre un escenario cuyas posiciones vienen dadas por el sistema cartesiano, calculando posiciones y giros de objetos.

Para conseguir este objetivo general, se marcan los siguientes objetivos específicos:

- Evaluar los efectos en la competencia de razonamiento abstracto de los alumnos después de la realización de la unidad didáctica de programación.
- Evaluar los efectos en la competencia de visión espacial de los alumnos después de la realización de la unidad didáctica de programación.
- Evaluar los efectos en la competencia de aptitud numérica de los alumnos después de la realización de la unidad didáctica de programación.

3.2 Instrumentos

Los instrumentos que se utilizarán para la realización de este estudio son los siguientes.

Por un lado, el test de Inteligencia General y Factorial IGF grado Elemental, que es adecuado para el curso de 1º de la ESO (Yuste-Hernanz, 1991). En este test se reflejan varios parámetros, entre los que destacamos el razonamiento abstracto, la aptitud espacial y la aptitud numérica, que son en los que nos centraremos en este estudio.

Por otro lado, la Unidad Didáctica personal que he realizado sobre Programación para 1º de la ESO, donde se describe el proyecto a realizar durante varias sesiones, trabajando en cada sesión diferentes grupos de bloques del entorno de Scratch (Domingo, 2016).

3.3 Cuestionarios

El test utilizado es el test de Inteligencia General y Factorial (IGF). Es un test normalizado y verificado por TEA, que no se puede publicar por derechos de autor (Yuste-Hernanz, 1991).

Dentro de este test hay varios grados, básico (B), elemental (E), medio (M) y superior (S), dependiendo de la edad a la que se vaya aplicar. En la siguiente tabla se puede ver la relación de test con los cursos a aplicar:

	PRIMARIA					ESO				BACHILLERATO		
	2º	3º	4º	5º	6º	1º	2º	3º	4º	1º	2º	Univ.
IGF-B	20' 22'	18' 20'	15' 16'	12' 13'								
IGF-E			40'	35'	30'	25'	25'					
IGF-M						40'	35'	35'	30'	30'		
IGF-S									45'	45'	40'	40'

Ilustración 6. Niveles del test IGF y cursos y tiempos de aplicación (Yuste-Hernanz, 1991)

Para realizar el proyecto se podrían utilizar el grado E o el grado M, pero se ha utilizado el IGF-E, es decir, el test de grado Elemental, que es adecuado para el curso de 1º de la ESO, curso donde se ha impartido la Unidad Didáctica de Programación y Scratch. El tiempo para la realización del test en este curso es de 25 minutos.

El test consta de 70 preguntas y mide los siguientes parámetros con su correspondiente significado. Es una herramienta que ha sido aplicada durante varios años en el Colegio San José en el Departamento de Orientación, obteniendo muy buenos resultados según los evaluadores y ha sido cedida para el objeto de esta investigación (Yuste-Hernanz, 1991):

Razonamiento Abstracto (RA):

Evalúa procesos de razonamiento y relaciones espaciales.

Una puntuación alta puede indicar:

- Buena capacidad para el razonamiento lógico.
- Agilidad mental y flexibilidad para resolver cuestiones de tipo lógico/abstracto.
- Facilidad para encontrar secuencias lógicas en figuras geométricas.

Se evalúa con las siguientes 21 preguntas del test: 1, 2, 3, 11, 12, 13, 21, 22, 23, 31, 32, 33, 41, 42, 43, 51, 52, 53, 61, 62 y 63.

Aptitud Espacial (ApE):

Es una habilidad para establecer relaciones de tamaño, distancia, dirección y forma, en una serie de figuras geométricas, para así, manejar figuras en el espacio reconociendo esas relaciones.

Una puntuación alta puede indicar:

- Facilidad para percibir diferencias en figuras en un espacio bi-dimensional.
- Habilidad para combinar figuras en el espacio reconociendo entre ellas una serie de relaciones de tamaño, distancia, posición y forma.
- Capacidad de girar imaginariamente figuras en el espacio conservando las nuevas relaciones de tamaño, forma y distancia establecidas al final de los giros.

Se evalúa con 14 preguntas del test, que son las siguientes: 4, 5, 14, 15, 24, 25, 34, 35, 44, 45, 54, 55, 64, 65.

Razonamiento Verbal (RV):

Es la capacidad para comprender verbalmente y razonar con contenidos verbales, así como relaciones entre conceptos.

Una puntuación alta puede indicar:

- Capacidad de encontrar relaciones entre palabras.
- Riqueza de vocabulario.

Se evalúa con las siguientes preguntas del test: 6, 7, 8, 16, 17, 18, 26, 27, 28, 36, 37, 38, 46, 47, 48, 56, 57, 58, 66, 67 y 68. Haciendo un total de 21 preguntas para evaluar esta competencia.

Aptitud Numérica (ApN):

Es un factor específico o aptitud para comprender y manejar símbolos matemáticos en la resolución de problemas.

Una puntuación alta puede indicar:

- Rapidez y seguridad en el cálculo mental.
- Buen reconocimiento de los símbolos aritméticos.
- Facilidad para resolver sencillos problemas numéricos.
- Comprensión de conceptos matemáticos.

Se evalúa con las siguientes 14 preguntas del test: 9, 10, 19, 20, 29, 30, 39, 40, 49, 50, 59, 60, 69 y 70.

Inteligencia No-Verbal (INV): $INV = RA + ApE$

Este factor refleja perfectamente la inteligencia fluida del sujeto, menos sensible al influjo cultural, más perceptivo/espacial y menos verbal.

Una puntuación alta puede indicar:

- Agilidad mental y flexibilidad para resolver cuestiones de tipo lógico-abstracto.
- Facilidad para comprender ideas o planteamientos que no están formulados con palabras.

Inteligencia Verbal (IV): $IV = RV + ApN$

Es una capacidad medida a través del lenguaje escrito, con símbolos lingüístico/matemáticos.

Una puntuación alta puede indicar:

- Buena inteligencia cultural.
- Buena comprensión verbal.
- Amplio conocimiento de conceptos verbales.
- Habilidad para captar relaciones entre conceptos.
- Riqueza de vocabulario.
- Habilidad para resolver problemas numéricos.

Inteligencia General (IG): $IG = INV + IV$

Engloba la diversidad de razonamientos lógicos, no-verbales, espaciales, verbales y numéricos.

Una puntuación alta puede indicar:

- Habilidad para la resolución de problemas de muy diverso tipo.
- Buena inteligencia tanto natural como adquirida.
- Gran flexibilidad intelectual para trabajar con contenidos mentales muy diversos.
- Buena capacidad para la deducción e inducción lógica.
- Rapidez y agilidad mental para captar relaciones y descubrir leyes o principios.

3.4 Muestra

El estudio se ha realizado en el Colegio San José de Getafe, en Madrid, a dos grupos de 1º de la ESO. El grupo A está compuesto de 26 alumnos y el grupo B de 30 alumnos.

3.5 Procedimiento

En primer lugar, se les realizó el test de inteligencia general y factorial en el grado elemental (IGF-E) antes de comenzar la unidad didáctica.

Durante la unidad didáctica de Programación en la asignatura de “Tecnología, Programación y Robótica” los alumnos estudiaron conceptos básicos de programación, incluyendo algoritmos y diagramas de flujo. Además, tuvieron que realizar un proyecto completo que consistía en el desarrollo del juego *Breakout* dentro del entorno de programación por bloques de Scratch, organizándose en grupos de 2 personas.

Durante la realización del proyecto, los alumnos tuvieron que crear un fondo de escenario y varios objetos (una bola, una paleta donde la bola debía rebotar, una persona o un animal para interactuar con el usuario y un detector de fallos en la parte inferior del escenario). Posteriormente dieron funcionalidad a cada uno de los objetos:

- La paleta debía moverse hacia la izquierda y hacia la derecha con el fin de que la bola chocase y así no permitirle caer por la parte inferior del escenario.
- La bola debía moverse continuamente por toda la pantalla, rebotar en los bordes del escenario y en la paleta y si tocaba el detector de fallos finalizaría el punto de la partida.
- Al finalizar el punto de la partida, debería aparecer la persona o el animal preguntando al usuario si quiere continuar o salir del juego.

- Si el usuario quiere continuar, la bola comienza a moverse de nuevo y la persona o animal se debe ocultar.
- Si el usuario quiere terminar el juego, se pararía despidiéndose la persona o animal.

Por último, tras finalizar la unidad didáctica, se les volvió a repetir el mismo test de inteligencia IGF-E.

4 Resultados

A continuación se mostrarán los resultados de los test, desglosándolos en Razonamiento Abstracto (RA), Aptitud Espacial (ApE) y Aptitud Numérica (ApN) para cada uno de los grupos.

4.1 Razonamiento Abstracto (RA)

A continuación se muestra el resultado del análisis del Razonamiento Abstracto del test previo y del test posterior a la Unidad Didáctica para los dos grupos:

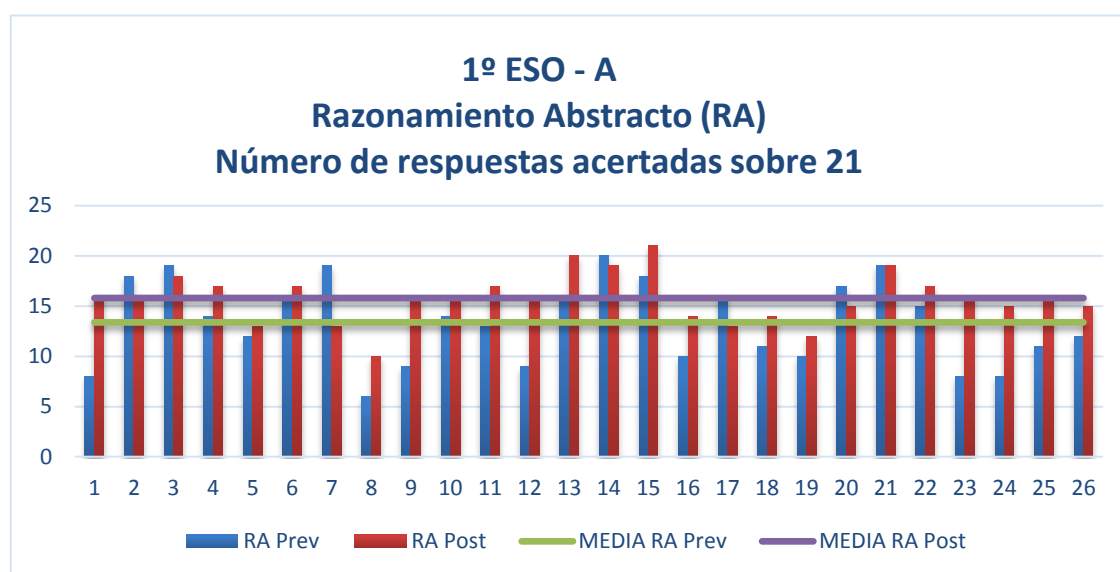


Ilustración 7. Evolución de la competencia de Razonamiento Abstracto en el grupo A. Fuente: Elaboración propia

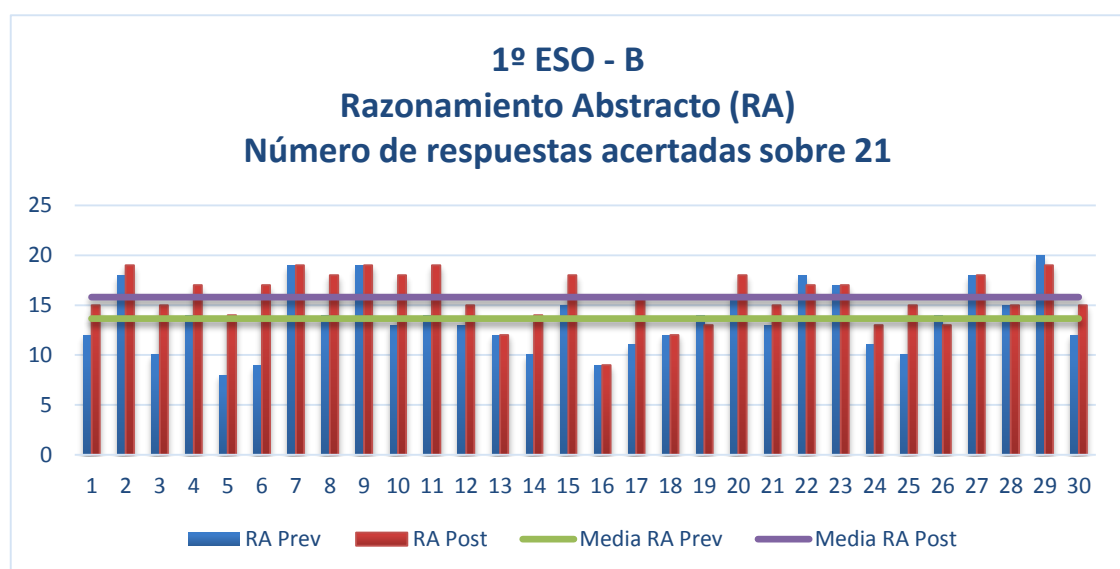


Ilustración 8. Evolución de la competencia de Razonamiento Abstracto en el grupo B. Fuente: Elaboración propia

En ambos grupos el resultado es muy parecido. Sobre un total de 21 respuestas, el número de las respuestas acertadas se ha visto incrementado en más de 2 tras la Unidad Didáctica.

	RAZONAMIENTO ABSTRACTO			
	1º ESO A		1º ESO B	
	RA Prev	RA Post	RA Prev	RA Post
1	8	16	12	15
2	18	16	18	19
3	19	18	10	15
4	14	17	14	17
5	12	13	8	14
6	16	17	9	17
7	19	13	19	19
8	6	10	14	18
9	9	16	19	19
10	14	16	13	18
11	13	17	14	19
12	9	16	13	15
13	16	20	12	12
14	20	19	10	14
15	18	21	15	18
16	10	14	9	9
17	16	13	11	16
18	11	14	12	12
19	10	12	14	13
20	17	15	16	18
21	19	19	13	15
22	15	17	18	17
23	8	16	17	17
24	8	15	11	13
25	11	16	10	15
26	12	15	14	13
27			18	18
28			15	15
29			20	19
30			12	15
MEDIA	13,38	15,81	13,67	15,80
VARIANZA	17,45	6,24	10,92	6,58

Tabla 1. Resultados del Razonamiento Abstracto en los test previo y posterior para los grupos A y B

En el grupo A la media inicialmente era de 13,38 respuestas acertadas (es decir, un 63,74%) con una dispersión de 17,45. Y en el test posterior a la unidad didáctica, se ha incrementado a 15,81 respuestas (es decir, un 75,27%) con una dispersión de 6,24.

En el grupo B la media en el test previo a la unidad didáctica era de 13,67 respuestas acertadas (es decir, un 65,08%) con una dispersión de 10,92 y se ha incrementado a 15,8 respuestas en el test posterior (es decir, un 75,24%) con una varianza de 6,58.

4.2 Aptitud Espacial (ApE)

Se muestran a continuación las gráficas de cada uno de los grupos que analizan la Aptitud Espacial según el test previo y el test posterior:

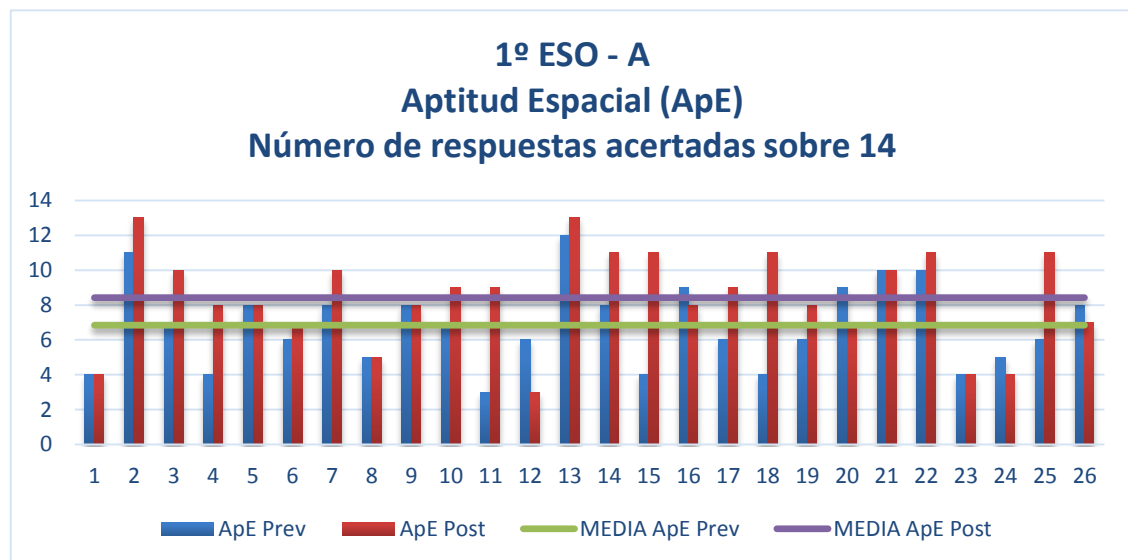


Ilustración 9. Evolución de la competencia de Aptitud Espacial en el grupo A. Fuente: Elaboración propia

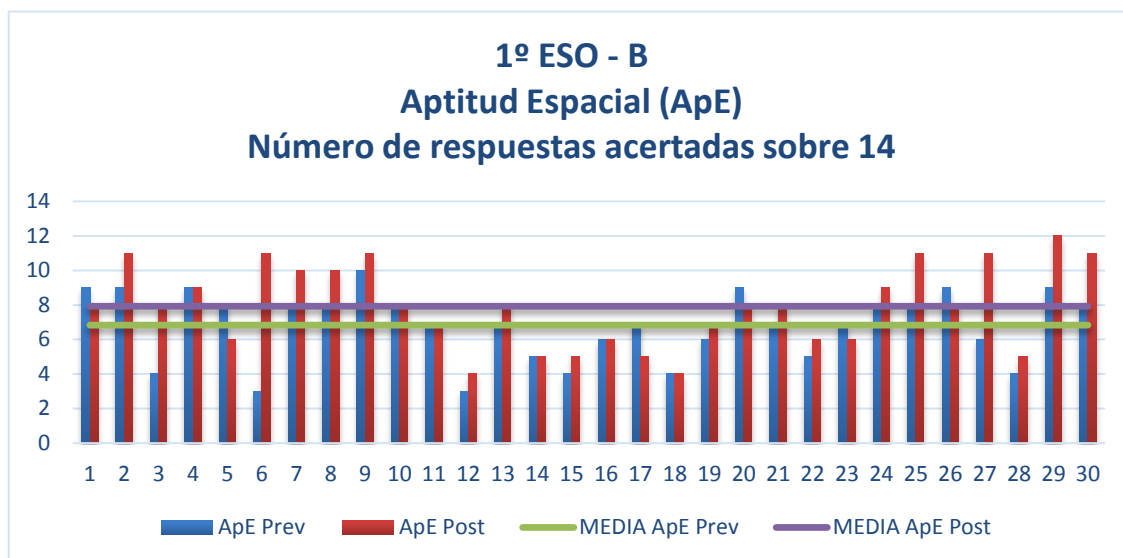


Ilustración 10. Evolución de la competencia de Aptitud Espacial en el grupo B. Fuente: Elaboración propia

Al analizar la competencia de Aptitud Espacial, los resultados en ambos grupos también son similares. La Aptitud Espacial dentro del test contiene un total de 14 preguntas. Y se puede apreciar en los gráficos cómo en general se ha visto incrementado en más de 1 respuesta acertada.

Así podemos verlo en la siguiente tabla donde aparecen el número de respuestas acertadas en el test previo y en el test posterior de todos los alumnos para los dos grupos, junto con la media aritmética y la varianza por grupo.

	APTITUD ESPACIAL			
	1º ESO A		1º ESO B	
	ApE Prev	ApE Post	ApE Prev	ApE Post
1	4	4	9	8
2	11	13	9	11
3	7	10	4	8
4	4	8	9	9
5	8	8	8	6
6	6	7	3	11
7	8	10	8	10
8	5	5	8	10
9	8	8	10	11
10	7	9	8	8
11	3	9	7	7
12	6	3	3	4
13	12	13	7	8
14	8	11	5	5
15	4	11	4	5
16	9	8	6	6
17	6	9	7	5
18	4	11	4	4
19	6	8	6	7
20	9	7	9	8
21	10	10	7	8
22	10	11	5	6
23	4	4	7	6
24	5	4	8	9
25	6	11	8	11
26	8	7	9	8
27			6	11
28			4	5
29			9	12
30			8	11
MEDIA	6,85	8,42	6,83	7,93
VARIANZA	5,82	7,61	4,07	5,72

Tabla 2. Resultados del Aptitud Espacial en los test previo y posterior para los grupos A y B

En el grupo A la media ha subido de 6,85 respuestas acertadas (es decir, un 48,9%) con una dispersión de 5,82 a 8,42 respuestas en el test posterior (es decir, un 60,16%) con una dispersión de 7,61.

En el grupo B la media ha subido de 6,83 respuestas acertadas (es decir, un 48,81%) con una dispersión de 4,07 a 7,93 respuestas en el test posterior (es decir, un 56,67%) con una dispersión de 5,72.

4.3 Aptitud Numérica (ApN)

A continuación se muestran las gráficas obtenidas de los test realizados a los dos grupos, en este caso midiendo la Aptitud Numérica, que consta de 14 preguntas dentro del test:

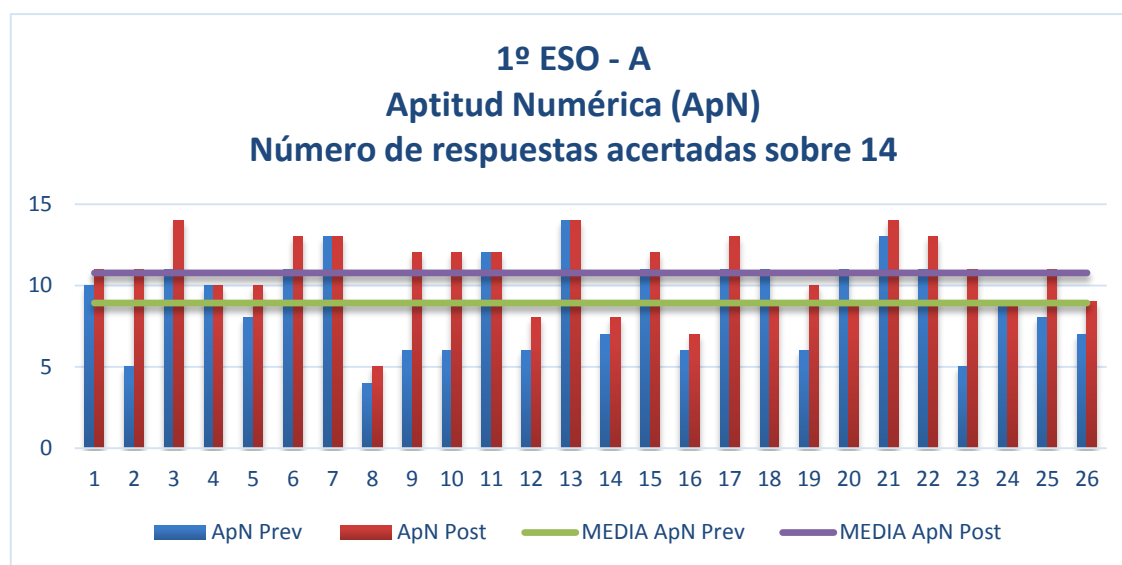


Ilustración 11. Evolución de la competencia de Aptitud Numérica en el grupo A. Fuente: Elaboración propia

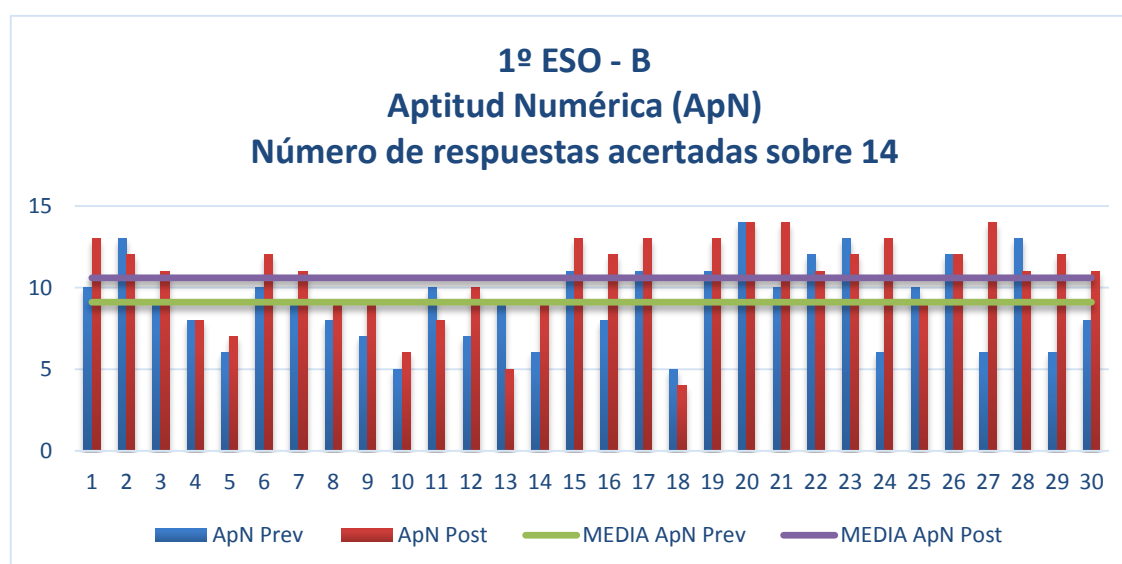


Ilustración 12. Evolución de la competencia de Aptitud Numérica en el grupo B. Fuente: Elaboración propia

Al analizar la competencia numérica, como en los casos anteriores, las gráficas demuestran que también se ha visto incrementada, en este caso en más de 1 respuesta correcta sobre el test previo a la unidad didáctica, sobre un total de 14 preguntas. Para ambos grupos los resultados han sido similares.

	APTITUD NUMÉRICA			
	1º ESO A		1º ESO B	
	ApN Prev	ApN Post	ApN Prev	ApN Post
1	10	11	10	13
2	5	11	13	12
3	11	14	9	11
4	10	10	8	8
5	8	10	6	7
6	11	13	10	12
7	13	13	9	11
8	4	5	8	9
9	6	12	7	9
10	6	12	5	6
11	12	12	10	8
12	6	8	7	10
13	14	14	9	5
14	7	8	6	9
15	11	12	11	13
16	6	7	8	12
17	11	13	11	13
18	11	9	5	4
19	6	10	11	13
20	11	9	14	14
21	13	14	10	14
22	11	13	12	11
23	5	11	13	12
24	9	9	6	13
25	8	11	10	9
26	7	9	12	12
27			6	14
28			13	11
29			6	12
30			8	11
MEDIA	8,92	10,77	9,10	10,60
VARIANZA	8,31	5,38	6,78	7,21

Tabla 3. Resultados del Aptitud Numérica en los test previo y posterior para los grupos A y B

En el caso del grupo A, la media se ha visto incrementada de 8,92 respuestas acertadas (es decir, un 63,74%) con una dispersión de 8,31 en el test previo a la unidad didáctica a 10,77 respuestas (es decir, un 76,92%) con una dispersión de 5,38 en el test posterior.

En el grupo B la media ha subido de 9,1 respuestas acertadas (es decir, un 65%) con una dispersión de 6,78 a 10,6 respuestas acertadas (es decir, un 75,71%) con una dispersión de 7,21 en el test posterior.

5 Conclusiones

5.1 Análisis de la experiencia

Es la primera vez que estos jóvenes han tenido contacto con la programación y, aunque no han tenido ningún miedo y les ha encantado jugar con el ordenador, no les ha resultado fácil empezar a pensar de una forma más estructurada de lo que lo hacemos habitualmente.

Antes de comenzar el proyecto, les resultó más sencilla la realización del algoritmo que la elaboración del diagrama de flujo, aunque al final de la unidad didáctica comprendieron perfectamente este concepto de diagrama de flujo.

Al comenzar a trabajar con el proyecto fue cuando comenzaron a ver algunas cosas más sencillas que no terminaban de comprender con los conceptos teóricos, como por ejemplo los bucles.

Durante la unidad didáctica y especialmente durante el proyecto han podido trabajar estos tres conceptos de Razonamiento Abstracto, Aptitud Espacial y Aptitud Numérica, que, como se ha visto en el apartado de Cuestionarios al analizar cada uno, tienen la siguiente relación con las habilidades STEM que se tratan en el estudio:

- Razonamiento Abstracto: equivale al razonamiento lógico-matemático, a la resolución de problemas y a la estructuración o desarrollo de ideas.
- Aptitud Espacial: equivale a la visualización gráfica y espacial y la visualización para girar imaginariamente figuras en el espacio, lo que significa tener un gran nivel de abstracción que permite mejorar la estructuración de ideas.
- Aptitud numérica: equivale a la resolución de problemas.

Por lo que mejorar estas 3 competencias, supone mejorar las siguientes habilidades STEM:

- Razonamiento lógico-matemático.
- Resolución de problemas.
- Estructuración de ideas o desarrollo de ideas a partir de una concepción inicial del proyecto.
- Visualización gráfica y espacial.

Por un lado, con la creación, la posición y el movimiento de los objetos que utilizaron en el proyecto de *Breakout* en Scratch, los alumnos estaban trabajando la aptitud espacial y la aptitud numérica.

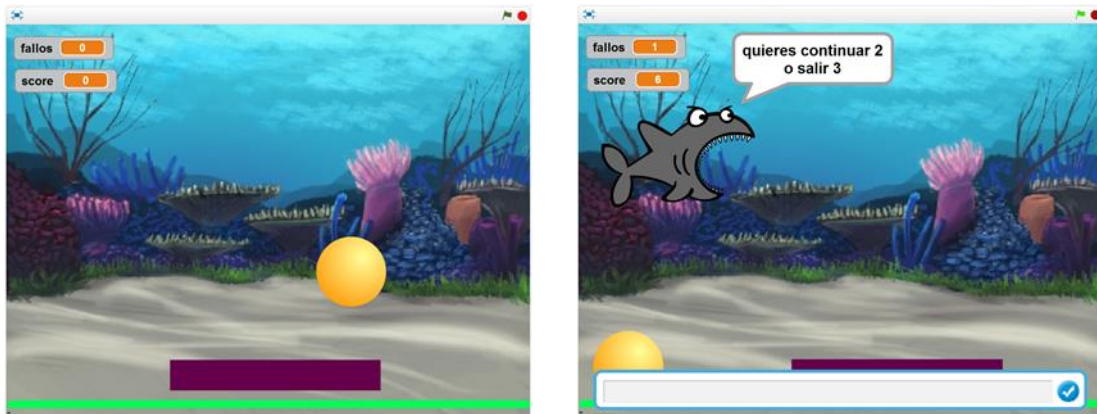


Ilustración 13. Imagen del proyecto de unos alumnos (Domingo, 2016)

Por otro lado, al hacer el algoritmo y programar por bloques cada uno de los objetos del proyecto, los alumnos estaban trabajando el razonamiento abstracto al pensar el funcionamiento de cada objeto, la aptitud espacial para controlar el movimiento de los objetos y la aptitud numérica, por ejemplo, para controlar el rebote en la paleta que fue una investigación de cada grupo ya que muchos de ellos lo obtuvieron sumando varios grados a la dirección que llevaba el objeto en ese momento.

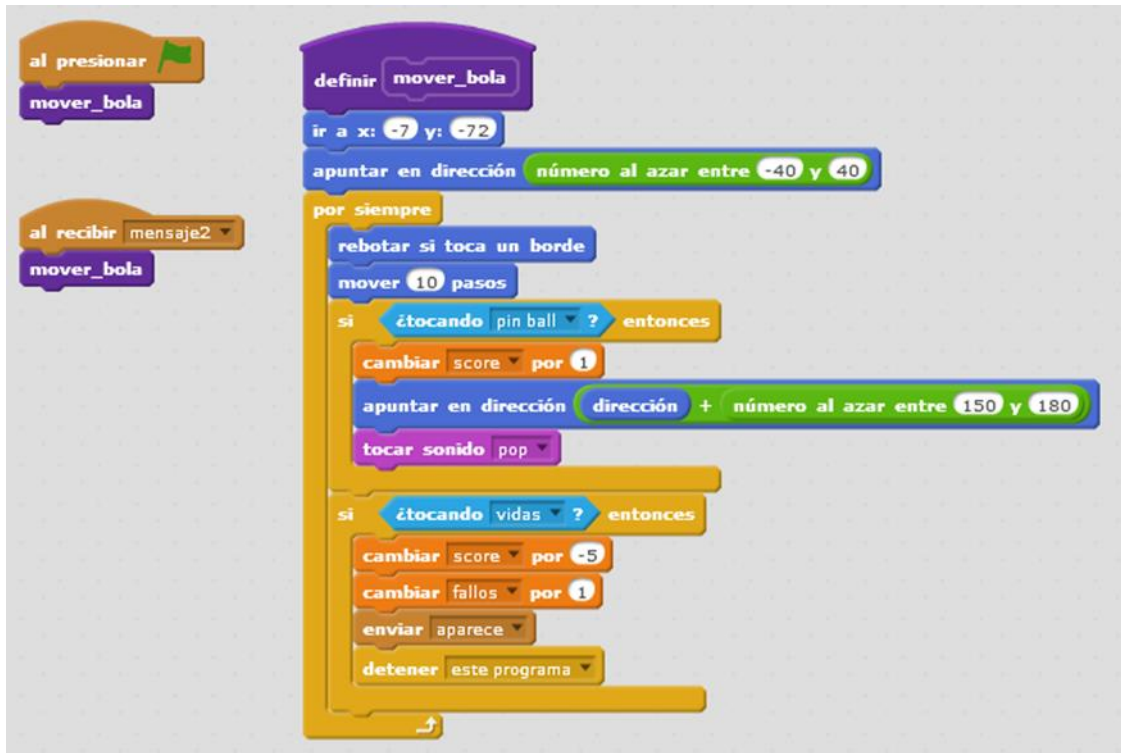


Ilustración 14. Programación del proyecto de unos alumnos (Domingo, 2016)

Hubo varios grupos que realizaron la versión avanzada del proyecto del Breakout:

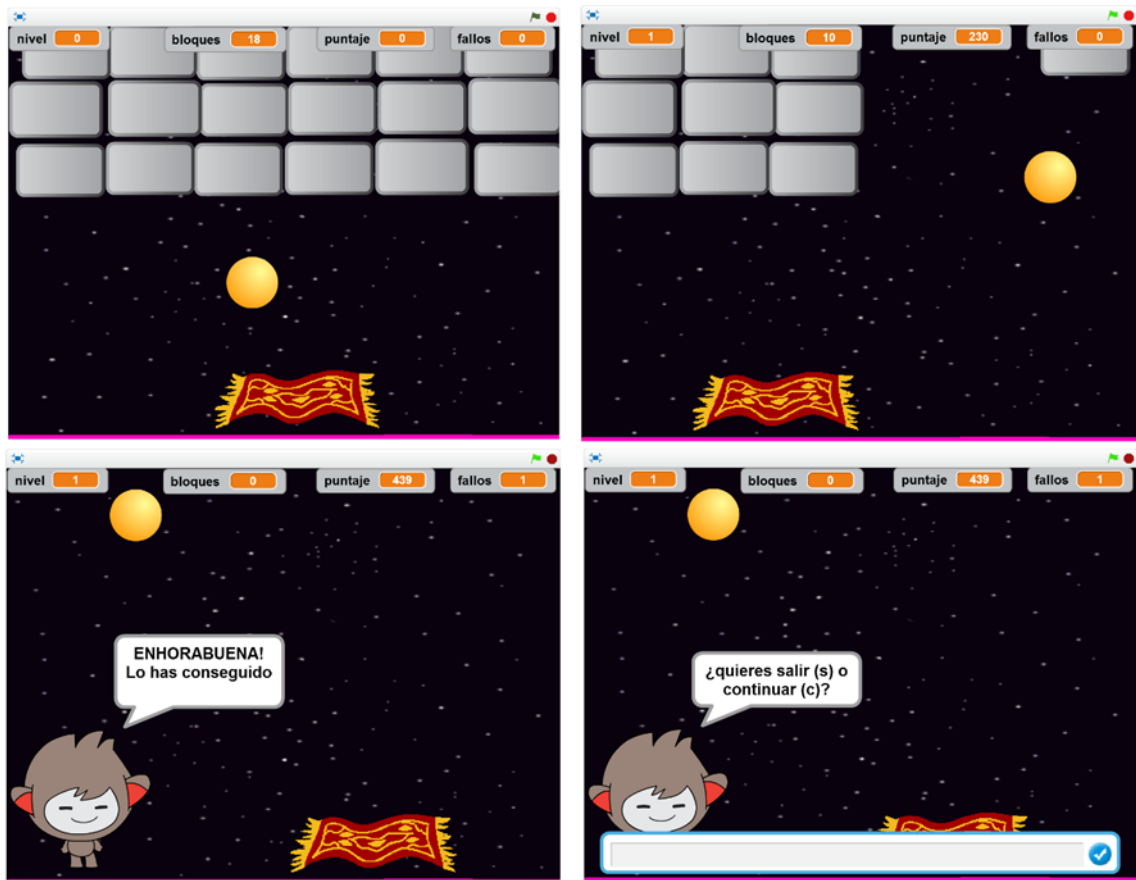


Ilustración 15. Versión avanzada del proyecto de un grupo de alumnos (Domingo, 2016)

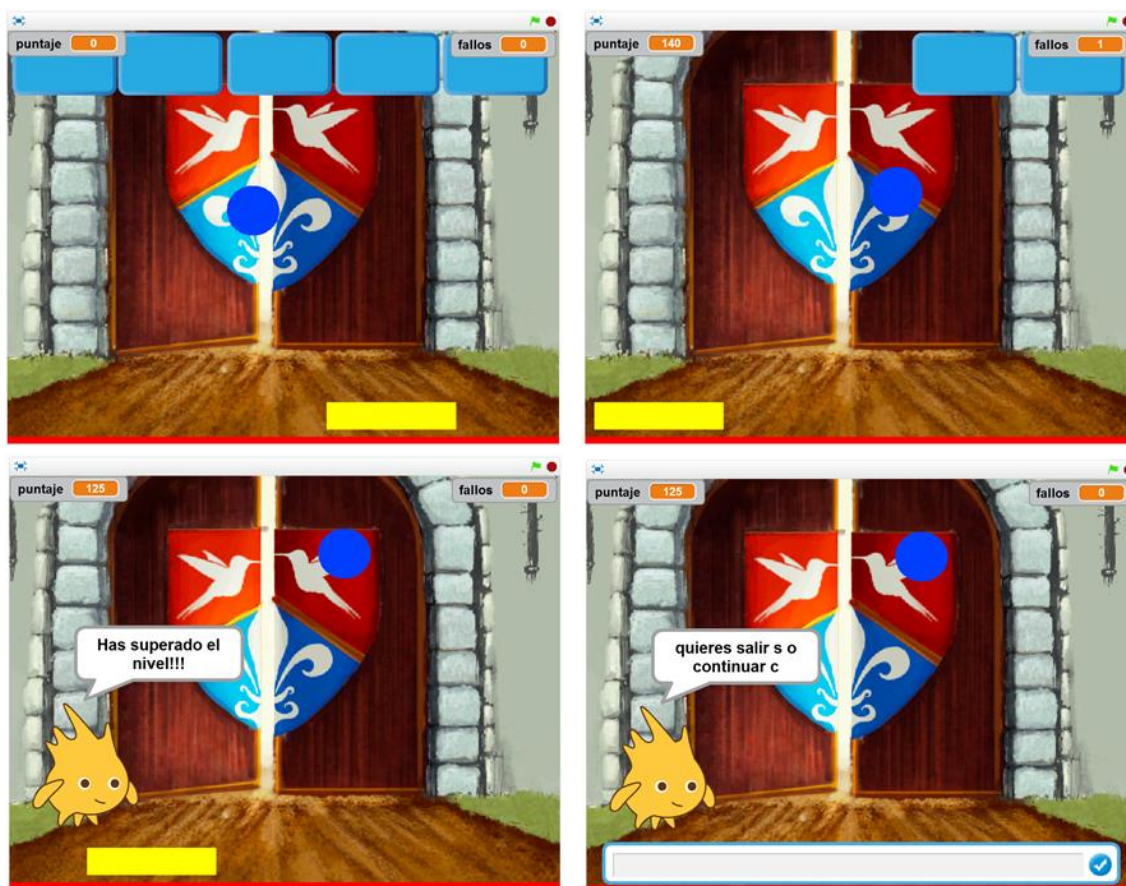


Ilustración 16. Versión avanzada del proyecto de un grupo de alumnos (Domingo, 2016)

Además, los resultados obtenidos en los test, como se puede apreciar en las gráficas anteriores, han sido positivos en las tres competencias analizadas.

Coincide, en los dos grupos, que los alumnos que han visto muy incrementadas las competencias de razonamiento abstracto y de aptitud espacial son alumnos que en general no van muy bien en el resto de las materias de clase, pero han estado ilusionados y motivados con Scratch, haciendo un trabajo mejor del esperado dada su actitud en el resto de las asignaturas. Es el caso de los números 4, 11, 15 y 18 en el grupo A y de los números 3 y 6 del grupo B. De hecho el caso del número 6 del grupo B es un poco peculiar, porque comenzó en el otro grupo y a mitad de curso le tuvieron que cambiar a este grupo dado su comportamiento y los resultados obtenidos han sido bastante favorables, como se puede comprobar en todas las pruebas.

5.2 Conclusiones finales

Con esta prueba, se puede afirmar que la programación hace que mejoren las competencias STEM, concretamente las competencias de razonamiento abstracto, de visión espacial y de aptitud numérica, analizadas en los resultados.

Y, dada la relación tan unida que tienen, de nuevo se puede afirmar que la programación mejora las competencias de razonamiento lógico-matemático, la resolución de problemas, la estructuración o desarrollo de ideas y la visualización gráfica y espacial.

6 Referencias bibliográficas

- ACM y CSTA (2010). Running on empty. Recuperado de <http://runningonempty.acm.org/fullreport2.pdf>.
- BCS The Chartered Institute for IT. (2016). *Computing at school*.
<http://www.computingschool.org.uk/>.
- Brennan, K. y Rusk, N. (2009). Scratch programming concepts. Recuperado de scratched.gse.harvard.edu/sites/default/files/scratchprogrammingconcepts-v14.pdf.
- Cano, E. V. y Delgado, D. F. (2015). La creación de videojuegos con Scratch en Educación Secundaria. *Communication Papers: Media Literacy and Gender Studies*, 4(6), 63-73.
- Cearreta-Urbieto, I. (2015). Scratch como recurso didáctico para el desarrollo del Pensamiento Computacional de los alumnos de Secundaria y Bachillerato en la asignatura de informática y como recurso transversal en el resto de asignaturas. Recuperado de <http://reunir.unir.net/handle/123456789/3150>.
- CODDII y AENUI. (2015). *Por la inclusión de asignaturas específicas de ciencia y tecnología informática en los estudios básicos de la enseñanza secundaria y bachillerato*. Recuperado de <http://coddii.org/wp-content/uploads/2015/06/declaraci%C3%B3n-CODDII-AENUI.pdf>.
- Domingo, M. (2016). *Programación. Unidad Didáctica 1º ESO*.
- Espeso, P. (2016). ¡Programa sin código con Scratch! Recuperado de <http://www.educaciontrespuntocero.com/recursos/programacion/resumen-reportaje-revista-scratch/35327.html>.
- Fundación Telefónica. (2014). *TOP 100 - innovaciones educativas*. Recuperado de https://publiadmin.fundaciontelefonica.com/index.php/publicaciones/add_descargas?tipo_fichero=pdf&idioma_fichero= &title=TOP+100+-+Innovaciones+Educativas&code=263&lang=es&file=Top100_es.pdf&_ga=1.42170533.1471571633.1462283701.
- Gander, W., Petit, A., Berry, G., Demo, B., Vahrenhold, J., McGettrick, A., . . . y Meyer, B. (2013). Informatics education: Europe cannot afford to miss the boat. *ACM* [Online] Recuperado de <http://www.informatics-europe.org/images/documents/informatics-education-acm-ie.pdf>.

- Gobierno de España (2013). Ley Orgánica 8/2013, de 9 de diciembre, para la Mejora de la Calidad Educativa. LOMCE. *Boletín Oficial del Estado*.
- Gobierno de la Comunidad de Madrid (2015). DECRETO 48/2015, de 14 de mayo, del Consejo de Gobierno, por el que se establece para la Comunidad de Madrid el Currículo de la Educación Secundaria Obligatoria. *Boletín Oficial de la Comunidad de Madrid*. Madrid.
- Instituto Nacional de Estadística. (2014, 02-10-2014). Encuesta sobre equipamiento y uso de Tecnologías de Información y Comunicación en los hogares. Año 2014. Recuperado de <http://www.ine.es/prensa/np864.pdf>.
- Instituto Nacional de Estadística. (2015, 01-10-2015). Encuesta sobre equipamiento y uso de Tecnologías de Información y Comunicación en los hogares. Año 2015. Recuperado de <http://www.ine.es/prensa/np933.pdf>.
- Lloyd, C. (2012, 03-12-2012). La importancia de las materias STEM en la escuela. Recuperado de <http://www.univision.com/noticias/educacion-primaria/la-importancia-de-las-materias-stem-en-la-escuela>.
- Maloney, J., Burd, L., Kafai, Y., Rusk, N., Silverman, B. y Resnick, M. (2004). Scratch: A sneak preview [education]. *Creating, Connecting and Collaborating through Computing, 2004. Proceedings. Second International Conference on* (pp. 104-109). IEEE.
- Meseguer, P., Moreno, J., Moreno, J. J., Olcoz, K., Pimentel, E., Toro, M., ... y Vendrell, E. Enseñanza de la informática en Primaria, Secundaria y Bachillerato: Estado Español, 2015.
- MIT. (2016). *Scratch*. <https://scratch.mit.edu/>.
- Montoya, R. S. TIC para estimular las Inteligencias. Recuperado de <http://www.educa2.madrid.org/web/educamadrid/principal/files/ef0ce754-7bf3-4d3a-ae0e-5db856adbe85/Discapacidad%20intelectual.pdf>.
- Naughton, J. (2012). A manifesto for teaching computer science in the 21st century. *The Observer*, 31.
- Quesada, J. F. (2015). Videojuegos matemáticos con Scratch. Recuperado de http://app.cepcastilleja.org/ebidegain/vi_encuentro/talleres/videojuegos.pdf.

- Randstad. (2016). *España será uno de los países europeos con mayor desajuste entre la formación de sus profesionales y las necesidades empresariales*. Recuperado de <http://www.randstad.es/nosotros/sala-prensa/Documents/objetivo-2020.pdf>.
- Resnick, M. y Brennan, K. (2011). Computational thinking practices. Recuperado de <http://scratched.gse.harvard.edu/resources/computational-thinking-practices-april-2011-webinar>.
- Resnick, M. (2007). Sembrando las semillas para una sociedad más creativa. *Laboratorio De*. Recuperado de http://robocamp.es/pdf/castellano/SEMBRANDO_LAS_SEMILLAS_PARA_UNA_SOCIEDAD_MAS_CREATIVA.pdf.
- UK Department of Education (2013). National curriculum in England: Computing Programmes of Study, DFE-00171-2013. Recuperado de <https://www.gov.uk/government/publications/national-curriculum-in-england-computing-programmes-of-study/national-curriculum-in-england-computing-programmes-of-study>.
- Uzcanga, I. y Gómez, M. Llevando las Ciencias, la Ingeniería, la Tecnología y la Matemática a la Escuela: Pequeños Científicos.
- Valle, J. E. M. y Salgado, V. C. (2013). Pensamiento lógico matemático con Scratch en nivel básico. *Vínculos*, 9(1), 87-95.
- Vásquez, A. L. (2015). Pensamiento computacional y la educación. Recuperado de <http://geneticcontent.blogspot.com.es/2015/06/pensamiento-computacional-y-la-educacion.html>.
- Wikipedia. (2016). Alan Turing. Recuperado de https://en.wikipedia.org/wiki/Alan_Turing.
- Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, 49(3), 33-35.
- Yuste-Hernanz, C. (1991). *IGF: Inteligencia general y factorial. Manual* (TEA ed.) Madrid.

7 Anexos

7.1 Unidad didáctica

UNIDAD DIDÁCTICA DE PROGRAMACIÓN

ÁREA O MATERIA	CURSO	TÍTULO DE LA UNIDAD DIDÁCTICA
Tecnología 1º ESO	1º ESO	Programación

Temporalización: 50 minutos/sesión Número de sesiones: 8

DESCRIPCIÓN
La Unidad Didáctica a desarrollar recibe el nombre de “Programación”. Esta unidad didáctica se encuentra en la asignatura de Tecnología para 1º ESO. En esta Unidad Didáctica, desarrollada en 8 sesiones, se impartirán los conceptos básicos sobre programación, qué son los programas y los lenguajes de programación que existen, qué son los algoritmos y los diagramas de flujo. Y por otro lado qué es Scratch y cómo trabajar con Scratch, mostrando su entorno de trabajo, los objetos y los bloques que contiene.

OBJETIVOS GENERALES
OG1. Analizar los diferentes niveles de lenguajes de programación OG2. Utilizar con destreza un entorno de programación gráfica por bloques OG3. Desarrollar y programar aplicaciones sencillas en entornos de programación por bloques

OBJETIVOS ESPECÍFICOS DE LA UNIDAD DIDÁCTICA
OE1. Identificar las características de los lenguajes de programación de bajo nivel. OE2. Identificar las características de los lenguajes de programación de alto nivel. OE3. Analizar el comportamiento de los programas a partir de sus diagramas de flujo. OE4. Analizar algoritmos OE5. Emplear, con facilidad, las diferentes herramientas básicas del entorno de programación en Scratch. OE6. Situar y mover objetos en una dirección dada en Scratch. OE7. Reconocer la forma de iniciar y detener la ejecución de un programa en Scratch. OE8. Modificar, mediante la edición, la apariencia de objetos en Scratch y crear nuevos objetos: disfraces, fondos y sonidos. OE9. Manejar, con soltura, los principales grupos de bloques del entorno en Scratch.

- OE10. Utilizar, con facilidad, los comandos de control de ejecución en Scratch: condicionales y bucles.
- OE11. Emplear de manera adecuada variables y listas en Scratch.
- OE12. Usar, con soltura, la interacción entre los elementos de un programa.

CRITERIOS DE EVALUACIÓN	OBJETIVO ESPECÍFICO
CE1. Analiza los diferentes niveles de lenguajes de programación	OE1, OE2
CE2. Interpreta el comportamiento de programas informáticos a partir de un algoritmo o de un diagrama de flujo	OE3, OE4
CE3. Utiliza con destreza un entorno de programación gráfica por bloques	OE5, OE6, OE7, OE8, OE9, OE10, OE11, OE12
CE4. Desarrolla y programa aplicaciones sencillas en entornos de programación por bloques	OE5, OE6, OE7, OE8, OE9, OE10, OE11, OE12

CONTENIDOS
• El lenguaje de los computadores ○ Programas informáticos ○ Lenguajes de programación ▪ Lenguajes de programación de bajo nivel ▪ Lenguajes de programación de alto nivel ○ Algoritmos ○ Diagramas de flujo • Programando en Scratch ○ Introducción ○ El entorno Scratch ○ Objetos: Propiedades, Disfraces y Sonidos ○ Bloques ○ Eventos ○ Operadores ○ Movimiento ○ Control ○ Apariencia ○ Sensores ○ Sonido ○ Lápiz ○ Datos

- Más bloques

CRONOGRAMA GENERAL					
	Sesión 1	Sesión 2	Sesión 3	Sesión 4	Sesión 5
El lenguaje de los computadores					
Scratch: Entorno					
Scratch: Bloques					
Scratch: Eventos					
Scratch: Operadores					
Scratch: Movimiento					
Scratch: Control					
Scratch: Apariencia					
Scratch: Sensores					
Scratch: Sonido					
Scratch: Lápiz					
Scratch: Datos					
Scratch: Más Bloques					

ESTRATEGIAS METODOLÓGICAS
El sistema metodológico del profesor se basa en la exposición, el trabajo personal de alumno y el trabajo en equipo. Por ello, utilizaremos las siguientes estrategias metodológicas: Técnica expositiva Se utilizará la técnica expositiva para presentar los contenidos de forma clara y ordenada, teniendo en cuenta las características de los alumnos y los conocimientos previos que tienen, con el fin de facilitar la información, motivando a los alumnos y fijando el contenido esencial o bien con resúmenes o repeticiones. Esta técnica es muy útil en este caso, donde todo el contenido es nuevo para los alumnos. También se utilizará para hacer repases de sesiones anteriores, para hacer resúmenes o para explicar las actividades que deben ir realizando. Técnica de la pregunta Para complementar la técnica expositiva y posibilitar un mejor aprendizaje, se utiliza la técnica de la pregunta. Se utiliza principalmente con el fin de que los alumnos participen en clase y reflexionen sobre algunos conceptos, además de usarlo también para retener conceptos mediante repaso. Cuchicheo y bola de nieve El cuchicheo hace que los alumnos, en grupos de dos, propongan una solución a un problema o alguna pregunta que plantee el docente. La bola de nieve es una variante del Cuchicheo que consiste en agrupar las parejas, quedando primero grupos de cuatro personas, y se puede ir ampliando después con

grupos de ocho, grupos de dieciséis,... Como finalidad tiene informar a los nuevos compañeros las conclusiones, las dudas o los puntos debatidos en el grupo anterior. Con estas dos técnicas se estimula la participación de todos los componentes del grupo y además, sirve para cambiar el ritmo ante una actividad monótona, por ejemplo es interesante alternarla con la técnica expositiva.

Phillips 66'

Primeramente, cada alumno piensa individualmente la solución a un problema o a una pregunta planteada por el docente. Después, se divide la clase en grupos de 6 personas para discutir el tema durante 6 minutos, haciendo que cada componente del grupo hable 1 minuto proponiendo su solución. Finalmente, trascurrido el tiempo, cada grupo expone sus conclusiones en voz alta.

Método de proyectos

El docente elige un tema y se formulan objetivos. Estructura el desarrollo del plan a realizar, tratando de aplicar en proyectos reales las habilidades y los conocimientos adquiridos.

En esta unidad didáctica, el proyecto se distribuye en fases que determina el docente a lo largo de varias sesiones. Se realiza en grupos de 2 personas y es el mismo para todos los grupos.

ACTIVIDADES	AGRUPAMIENTO	COMPETENCIAS BÁSICAS	OBJETIVOS ESPECÍFICOS
Realización grupal de un algoritmo	Primero 2 y después 4	CB1, CB2, CB4, CB5, CB6	OE4
Realización individual primero y grupal después de un diagrama de flujo	Primero individual y después por parejas	CB1, CB2, CB4, CB5, CB6	OE3
Elaboración de un proyecto para desarrollar un juego en Scratch	Por parejas	CB1, CB2, CB3, CB4, CB5, CB6	OE5, OE6, OE7, OE8, OE9, OE10, OE11, OE12
Elaboración de una memoria del proyecto individual donde se indique qué han hecho en la práctica	Individual	CB1, CB6, CB7	OE4
Examen Kahoot tipo test sobre la unidad didáctica de programación	Individual	CB3	OE1, OE2
Co-evaluación del proyecto	Por parejas	CB1, CB2, CB4, CB6	OE7

Examen de preguntas cortas sobre la unidad didáctica de programación	Individual	CB1, CB2, CB6, CB7	OE1, OE2, OE3, OE4, OE5, OE6, OE7, OE8, OE9, OE10, OE11, OE12
--	------------	--------------------	---

Las competencias básicas son las marcadas por la LOMCE:

CB1. Competencia en comunicación lingüística

CB2. Competencia matemática y competencias básicas en ciencia y tecnología

CB3. Competencia digital

CB4. Aprender a aprender

CB5. Competencias sociales y cívicas

CB6. Sentido de la iniciativa y espíritu emprendedor

CB7. Conciencia y expresiones culturales

CRONOGRAMA DE ACTIVIDADES								
	SESIONES							
ACTIVIDADES	1	2	3	4	5	6	7	8
Realización grupal de un algoritmo								
Realización individual primero y grupal después de un diagrama de flujo								
Elaboración de una práctica para desarrollar un juego en Scratch								
Elaboración de una memoria de prácticas individual donde se indique qué han hecho en la práctica								
Examen Kahoot tipo test sobre la unidad didáctica de programación								
Co-evaluación del proyecto								
Examen de preguntas cortas sobre la unidad didáctica de programación								

RECURSOS DIDÁCTICOS
Todas las sesiones, salvo la primera y la última, se impartirán en el aula de informática, ya que es necesario un ordenador para cada 2 alumnos. Los alumnos contarán con unos apuntes de elaboración propia que les enviaré a través de la plataforma educativa del Centro. En dichos apuntes se incluye bibliografía que pueden utilizar siempre que lo deseen.

EVALUACIÓN	PORCENTAJE
Observación (actitud, cuaderno de clase, trabajo en equipo,...) y otras actividades (realización de algoritmo y realización de diagrama de flujo)	5%
Elaboración de un proyecto para desarrollar un juego en Scratch	40%
Elaboración de una memoria de proyecto individual donde se indique el grado de implicación en la práctica y lo que han hecho	15%
Examen Kahoot tipo test sobre la unidad didáctica de programación	5%
Co-evaluación del proyecto (evaluación de la forma de evaluación a los compañeros)	5%
Prueba escrita de preguntas cortas sobre la unidad didáctica de programación	30%

DESARROLLO DE LAS SESIONES DE AULA

(ESTRATEGIAS METODOLÓGICAS Y LAS ACTIVIDADES A REALIZAR)

La unidad didáctica se desglosa en 8 sesiones de 50 minutos, que se dividirán de la siguiente forma:

DESGLOSE DE SESIONES	
SESIÓN 1	Lenguaje de los computadores
SESIÓN 2	Scratch: Entorno y Objetos
SESIÓN 3	Scratch: Bloques, Eventos, Movimiento y Operadores
SESIÓN 4	Scratch: Control, Apariencia y Sensores
SESIÓN 5	Scratch: Sonidos, Lápiz, Datos y Más bloques
SESIÓN 6	Scratch: Proyecto
SESIÓN 7	Scratch: Entrega del proyecto y Co-evaluación
SESIÓN 8	Evaluación

SESIÓN 1

1. (10 min.) Técnica expositiva

Presentación sobre qué es la programación, los lenguajes de programación que hay y qué es un algoritmo

Técnica de la pregunta: los alumnos deben ir diciendo cómo sería el algoritmo de cepillarse los dientes, puesto de ejemplo en la parte expositiva

2. (5 min.) Cuchicheo

Los alumnos deben debatir en parejas cómo sería el algoritmo para hacer una Tortilla de Patatas

3. (10 min.) Bola de nieve

Se unen dos parejas para ayudarse y comentar cómo es su algoritmo para hacer una tortilla de patatas.

Posteriormente, se exponen las conclusiones mostrando una posible solución en voz alta de cómo sería el algoritmo para hacer una tortilla de patatas

4. (5 min.) Técnica expositiva

Presentación sobre qué es un diagrama de flujo y cómo se realiza

5. (20 min.) Phillips 66'

Los alumnos en primer lugar deben pensar individualmente cómo sería el diagrama de flujo para montar una mesa de Ikea. Después, se organizan en grupos de 6 personas para debatir cómo sería dicho diagrama de flujo. Y, transcurrido el tiempo, cada grupo muestra una posible solución en voz alta.

SESIÓN 2

1. **(5 min.) Ir a la sala de informática**
2. **(5 min.) Organización de los grupos de trabajo**
Se organizarán grupos de trabajo de 2 personas
3. **(5 min.) Técnica expositiva**
Explicación del entorno de Scratch y de cómo darse de alta en la web de Scratch.
4. **(5 min.) Método de proyectos**
Uno de los miembros se tiene que dar de alta en la web de Scratch
5. **(10 min.) Técnica expositiva**
Explicación de la pantalla principal de Scratch, de la barra de menús, del escenario y de los objetos (programas, disfraces y sonidos)
6. **(5 min.) Técnica expositiva**
Explicación del proyecto que van a tener que ir realizando a lo largo de las sesiones de esta unidad didáctica. Se mostrará el juego completo con lo que deben hacer (Breakout) y se les dará una guía con los pasos que deben seguir durante esta sesión:
 - Crear un fondo de escenario
 - Crear un objeto paleta
 - Crear un objeto bola
 - Crear un objeto persona o animal
 - Crear un objeto detector de fallos
7. **(10 min.) Método de proyectos**
Cada grupo trabajará en el proyecto según las indicaciones dadas en la técnica expositiva de explicación del proyecto.
8. **(5 min.) Recoger para volver a clase**

SESIÓN 3

1. **(5 min.) Ir a la sala de informática**
2. **(5 min.) Técnica expositiva**
Breve repaso de lo visto el día anterior
Técnica de la pregunta: preguntar a los alumnos si tienen alguna duda
3. **(10 min.) Técnica expositiva**
Explicación de los tipos de bloques que hay en Scratch y explicación de los bloques de eventos, de movimiento y de operadores
Técnica de la pregunta: preguntar a los alumnos qué resultados darían los diferentes operadores lógicos, o los operadores con texto
4. **(5 min.) Técnica expositiva**
Recordar a los alumnos lo que deberían tener hecho de la semana anterior y mostrar una guía con los pasos que deben seguir durante esta sesión para avanzar en el proyecto que van a realizar a lo largo de las sesiones de la unidad didáctica:
 - El objeto paleta se debe mover a la derecha y a la izquierda al utilizar las flechas del teclado y debe rebotar si toca un borde

- El objeto bola se debe mover continuamente y debe rebotar si toca algún borde
 - El objeto persona o animal debe estar oculto al iniciar el programa
5. **(20 min.) Método de proyectos**
Cada grupo trabajará en el proyecto según las indicaciones dadas en la técnica expositiva de explicación del proyecto.
6. **(5 min.) Recoger para volver a clase**

SESIÓN 4

1. **(5 min.) Ir a la sala de informática**
2. **(5 min.) Técnica expositiva**
Breve repaso de lo visto el día anterior
Técnica de la pregunta: preguntar a los alumnos si tienen alguna duda
3. **(10 min.) Técnica expositiva**
Explicación de los bloques de control, de apariencia y de sensores
Técnica de la pregunta: relacionar los bloques de control con los diagramas de flujo
4. **(5 min.) Técnica expositiva**
Recordar a los alumnos lo que deberían tener hecho en semanas anteriores y mostrar una guía con los pasos que deben seguir para avanzar en el proyecto que van a realizar a lo largo de las sesiones de esta unidad didáctica:
- El objeto bola
 - si toca la paleta tiene que apuntar en dirección “dirección + un número al azar entre 150 y 180) y que siga moviéndose
 - si toca el detector de fallos tiene que enviar un evento de “fin de punto” al objeto persona/animal
 - El objeto persona o animal
 - Al recibir el evento “fin de punto” tiene que aparecer en pantalla preguntando si quiere salir o continuar
 - Si quiere salir, debe mostrar un mensaje para que el usuario vuelva pronto, se debe esconder y detener todos los programas
 - Si quiere continuar, debe mostrar un mensaje alegrándose porque siga jugando, debe enviar un evento “nuevo punto”, se debe esconder y después detener su propio programa
5. **(20 min.) Método de proyectos**
Cada grupo trabajará en el proyecto según las indicaciones dadas en la técnica expositiva de explicación del proyecto.
6. **(5 min.) Recoger para volver a clase**

SESIÓN 5

1. (5 min.) Ir a la sala de informática

2. (5 min.) Técnica expositiva

Breve repaso de lo visto el día anterior

Técnica de la pregunta: preguntar a los alumnos si tienen alguna duda

3. (10 min.) Técnica expositiva

Explicación de los bloques de sonido, de lápiz, de datos y explicación de la opción que tiene Scratch para incluir más bloques

Técnica de la pregunta: preguntar a los alumnos qué es una constante, qué es una variable y qué es una lista

4. (10 min.) Técnica expositiva

Recordar a los alumnos lo que deberían tener hecho de la semana anterior y mostrar una guía con los pasos que deben seguir durante esta sesión para avanzar en el proyecto que van a realizar a lo largo de las sesiones de esta unidad didáctica:

- Comenzar de nuevo la partida al recibir el evento “nuevo punto”, es decir, la bola debe empezar a moverse de nuevo y, si se desea, colocar la paleta en el centro
- Añadir una variable “fallos” (visible desde el programa) que tiene que incrementarse cuando la bola toca en el detector de fallos
- El objeto bola si toca la paleta, debe emitir un sonido y si toca el detector de fallos debe sonar un instrumento musical

Versión avanzada de la práctica:

- Crear n objetos caja_1,..., caja_n, donde $n \geq 5$
- Añadir una variable que indique el número de bloques que se muestran en la pantalla. Inicialmente, se muestran todos, es decir, “número de bloques” = n
- Si la bola toca un objeto de caja, la bola rebota, dicho objeto desaparece de la pantalla y el número de bloques disminuye
- Cuando todos los bloques desaparezcan, sale la persona/animal preguntando si quiere continuar la partida o quiere salir

Opciones de mejora:

- Añadir 3 vidas y que la partida se acabe cuando las vidas se agoten
- Añadir niveles
- Aumentar la dificultad del juego con los niveles, por ejemplo, que aumente la velocidad de la bola

5. (20 min.) Método de proyectos

Cada grupo trabajará en el proyecto según las indicaciones dadas en la técnica expositiva de explicación del proyecto.

6. (5 min.) Recoger para volver a clase

SESIÓN 6

1. (5 min.) Ir a la sala de informática

2. (5 min.) Test Kahoot

Realización de un test online con el fin de que ellos vean cómo van de preparados para el examen de preguntas de la siguiente sesión.

3. (5 min.) Técnica expositiva

Breve repaso de lo que tienen que hacer para la práctica. Explicarles que tienen que entregarlo para la siguiente semana y que además deben entregar un pequeño documento individual indicando qué es lo que han realizado en la práctica.

4. (30 min.) Método de proyectos

Cada grupo trabajará en el proyecto según las indicaciones dadas en la técnica expositiva de explicación del proyecto.

5. (5 min.) Recoger para volver a clase

SESIÓN 7

1. (5 min.) Ir a la sala de informática

2. (5 min.) Repaso

Para finalizar el tema y antes de la evaluación, haremos un pequeño repaso a todo lo que hemos visto

3. (5 min.) Dudas y preguntas

Se aclararán las dudas y todas las preguntas que pudiesen surgir sobre todo el contenido visto en la unidad didáctica de programación.

4. (30 min.) Co-evaluación

Deben evaluar a otro grupo de la clase según la rúbrica establecida para el proyecto.

5. (5 min.) Recoger para volver a clase

SESIÓN 8

1. (25 min.) Evaluación

Evaluación de la unidad didáctica. El examen es de preguntas cortas, podría ser tipo test, pero el objetivo es conseguir que los alumnos aprendan a pensar.

2. (20 min.) Revisión de los mejores proyectos

Visualización de los proyectos que me han parecido interesantes, por su originalidad y su funcionalidad.

3. (5 min.) Visualización de un video sobre nuevas tecnologías (Impresoras 3D)

Visualización de un vídeo de elaboración propia sobre Impresoras 3D. El objetivo es hacer ver a los alumnos hasta dónde puede llegar la tecnología.

La evaluación del proyecto se llevará a cabo con la siguiente rúbrica:

	0	1	2
Creación de objetos	No ha creado correctamente el escenario o los objetos paleta, bola, persona/animal y detector de fallos	Ha creado correctamente el escenario y los objetos paleta, bola, persona/animal y detector de fallos	VERSIÓN AVANZADA Además de crear los objetos paleta, bola, persona/animal y detector de fallos, también ha creado los objetos "caja_1",... "caja_n"
Funcionalidad de la paleta	La paleta no se mueve correctamente hacia la derecha o hacia la izquierda	La paleta se mueve correctamente hacia la derecha y hacia la izquierda	La paleta se mueve correctamente hacia la izquierda y hacia la derecha y rebota en los bordes
Funcionalidad de la bola	La bola no se mueve continuamente o no rebota en los bordes o no rebota en la paleta	La bola se mueve continuamente y rebota en los bordes y rebota en la paleta	La bola se mueve continuamente y rebota en los bordes y rebota en la paleta y emite un sonido al tocar en la paleta
Funcionalidad de la bola y detector de fallos	Cuando la bola toca el detector de fallos no se para	Cuando la bola toca el detector de fallos, se para	Cuando la bola toca el detector de fallos, suena un instrumento, aumenta el número de fallos (o disminuye el número de vidas) y se para.
Funcionalidad de la persona/animal	La persona/animal está oculta al comienzo del juego	La persona/animal está oculta al comienzo del juego pero no aparece preguntando si quiere continuar o salir cuando la bola toca el detector de fallos	La persona/animal está oculta al comienzo del juego y aparece preguntando si quiere continuar o salir cuando la bola toca el detector de fallos
Funcionalidad tras responder	El juego no continúa si la respuesta a la pregunta es "Continuar" o el juego no se para si la respuesta es "Salir"	El juego se para si la respuesta es "Salir" y continúa si la respuesta a la pregunta es "Continuar" (la bola se pone de nuevo en movimiento)	El juego se para si la respuesta es "Salir" y continúa si la respuesta a la pregunta es "Continuar" (la bola se pone de nuevo en movimiento) y la persona/animal desaparece
Versión avanzada	No ha implementado versión avanzada o los objetos "caja_i" no desaparecen cuando la bola los toca	VERSIÓN AVANZADA Si la bola toca los objetos "caja_i", estos objetos desaparecen	VERSIÓN AVANZADA Si la bola toca los objetos "caja_i", estos objetos desaparecen y se modifica la variable del "numero de bloques en pantalla"
Originalidad	No es original	Es original, se puede ver la creatividad del grupo	